# TailorTrack Tailoring Shop Management System

Web Programming Project Proposal | Fall 2026

## Problem statement

Tailoring shops often record customer measurements, orders and delivery dates in paper notebooks. This makes records difficult to retrieve and orders difficult to track. A small web system can bring customer details and order progress into one place.

## Proposed solution

Build an application for one tailoring shop. The shop owner manages customers and orders, while staff update the work assigned to them. Use saved order statuses and a simple dashboard to track progress and upcoming deliveries.

## Main requirements

**1 Accounts and permissions.** Provide login and logout for Owner and Staff. The owner can manage all records; staff can view and update only assigned orders. Prepare staff accounts separately rather than allowing users to choose the Owner role.

**2 Customers and measurements.** Create, view and edit customer profiles with name, phone number and a simple measurement form. Search by name or phone. Keep saved measurements available for future orders, archive customers with order history instead of deleting them.

**3 Orders and assignment.** Create an order linked to a customer, with garment type, measurements copied into the order, instructions, price, advance payment and due date. Assign it to a staff member. Editing the customer profile must not change previous orders.

**4 Order progress.** Use Pending, In progress, Ready and Delivered statuses. Staff can update assigned orders up to Ready; the owner records delivery. Show overdue unfinished orders and filter by status, staff member or due date.

**5 Payment record and dashboard.** Record payments manually and calculate the remaining balance. Show total active orders, ready orders and overdue orders. Include an order details page with a printable summary using browser printing.

**6 Garment photos and staff work summary.** Upload a garment reference photo with file type/size validation and authorized access. The owner can set a simple per-order work rate and view each staff member’s completed orders and earned total. No payroll or salary processing is needed.

## Technology and implementation requirements

- Frontend: React with HTML and CSS. Use reusable components, forms and a responsive layout for mobile and desktop. Bootstrap or Tailwind may be used; apply basic TypeScript types to your main data models.
- State and requests: use React hooks for local state and Redux for basic shared login state. Connect to the backend using fetch or axios with async/await. Show loading, empty, success and error states.
- Backend: Node.js and Express with REST APIs. Use suitable HTTP methods for reading, creating, updating and deleting records. Add middleware for login checks, permissions and error handling.
- Database: choose MongoDB or MySQL. Save related records and retrieve them through the backend. Data must survive refresh and application restart.
- Authentication and validation: provide cookies/sessions or a token-based login. Validate required fields and project rules on both frontend and backend. Never rely on hidden buttons alone to restrict access.

## Suggested database records

Users, Customers and Orders. Orders reference a customer and an assigned staff member and store their own measurements and payment entries. Store photo metadata and the staff work rate with each order.

## Scope and additional instructions

Validate positive prices, payment totals and allowed status changes. This project covers one shop, online payments, payroll, WhatsApp integration, voice recording and AI are not required.

Use fictional accounts and sample data. Password hashing and advanced security configuration are not additional grading requirements. Aim for a manageable set of screens rather than adding unrelated features.